

Conceptos aprendidos

El buen prompt

[1. Rol]

Actúa como un desarrollador backend Senior especializado en ciberseguridad.

[2. Contexto]

Estoy construyendo una API REST para un e-commerce usando Python 3.10.

[3. Tarea]

Necesito que escribas un endpoint para el login de usuarios.

[4. Restricciones]

El endpoint debe estar hecho con el framework FastAPI. Debe recibir un email y una contraseña, validar las credenciales simulando una consulta a una base de datos PostgreSQL y devolver un token JWT. Asegúrate de incluir el manejo de errores (por ejemplo, devolver un error 401 si las credenciales fallan) y usa contraseñas hashadas con bcrypt.

[5. Formato]

Devuélveme únicamente el bloque de código bien comentado, sin explicaciones previas ni introducciones.

<https://platform.openai.com/chat>
<https://aistudio.google.com>

NotebookLM

Cursor editor de código

Git Worktree

Kiro

Codez

AntiGravity

Vite

Vite es una herramienta de construcción (*build tool*) para el frontend diseñada para ofrecer una experiencia de desarrollo significativamente más rápida y moderna en comparación con los empaquetadores tradicionales.

Se utiliza principalmente por las siguientes razones:

1. Velocidad de inicio casi instantánea

A medida que las aplicaciones crecen, los empaquetadores tradicionales (*bundlers*) tardan mucho en procesar todo el código antes de mostrar algo en el navegador. **Vite cambia este enfoque dividiendo el trabajo:**

- **Dependencias:** Las bibliotecas externas que no cambian a menudo se **preempaquetan** una sola vez con herramientas nativas extremadamente rápidas (como esbuild), por lo que están listas al instante.

React Router

2. React Router: La pieza que falta en React

React se encarga de renderizar componentes en la pantalla, pero por sí solo no sabe cómo manejar los cambios de URL. **React Router** es la librería estándar que soluciona esto, permitiendo que tu app se comporte como una web tradicional con navegación fluida pero sin recargas de página.

Desde la versión **v7**, el paquete react-router-dom ha quedado obsoleto, integrándose todas las funcionalidades del DOM (como rutas y enlaces) directamente en el paquete principal react-router.

Express

JWT

Elige JWT si:

- Estás desarrollando **APIs para aplicaciones móviles**.
- Tu arquitectura utiliza **microservicios**, donde cada servicio puede verificar el token de forma independiente sin consultar una base de datos central.
- Necesitas **escalabilidad masiva** y quieres evitar la sobrecarga de consultas a la base de datos en cada petición

Bcrypt

Prisma

Basado en las fuentes, aquí tienes el esquema de flujo del proceso de funcionamiento de Prisma ORM:

Paso 1: Modelado de datos en el Prisma Schema (archivo .prisma), el cual actúa como la única fuente de verdad para definir las entidades y sus relaciones

-->

Paso 2: Sincronización mediante Prisma Migrate, que automatiza la creación y el seguimiento de los cambios estructurales en la base de datos real a partir del esquema definido

-->

Paso 3: Generación del Prisma Client, que crea una API a medida, con seguridad de tipos (type-safe) y autocompletado en el editor, basada específicamente en tus modelos de datos

-->

Paso 4: Ejecución de consultas (queries) desde el código de la aplicación (Node.js o TypeScript) utilizando los métodos del cliente generado para realizar operaciones CRUD

-->

Paso 5: Obtención de resultados en objetos JavaScript, donde Prisma traduce las filas de la base de datos relacional a objetos normales, permitiendo a los desarrolladores razonar sobre los datos de forma sencilla

Prisma Migrate

Prisma Migrate es una potente herramienta de migración de bases de datos integrada en Prisma ORM que se encarga de sincronizar automáticamente el modelo de datos definido en tu **Prisma Schema** con la estructura física de tu base de datos real. Como vimos anteriormente en nuestro flujo de proceso, actúa como el puente que transforma tus definiciones de objetos en cambios estructurales en el motor de base de datos.

Vecel

Render/Railway

Documentación

Anime Platform

Aplicación web para explorar anime, reproducir episodios con licencia y guardar el progreso de usuarios autenticados.

Requisitos

- Node.js 20+
- PostgreSQL 15+

Inicio rápido

1. Copia `server/.env.example` a `server/.env` y completa `DATABASE_URL` y `JWT_SECRET`.
2. Crea la base de datos indicada en `DATABASE_URL`.
3. Ejecuta `npm install` en `server` y en `client`.
4. En `server`, ejecuta `npx prisma migrate dev --name init` y después `npm run dev`.
5. En `client`, ejecuta `npm run dev`.

La API queda en `http://localhost:4000` y el cliente en `http://localhost:5173`.

> Solo configura URLs de vídeo de proveedores autorizados.

Capa	Tecnología	Justificación
Frontend	React + Vite + JavaScript + CSS	React facilita una interfaz dinámica; Vite ofrece desarrollo y compilación rápida.
Enrutamiento	React Router	Permite rutas protegidas para perfil, historial y reproductor.
Backend	Node.js + Express	Ligero, sencillo y usa JavaScript en todo el proyecto.
Autenticación	JWT + bcrypt	JWT protege las sesiones y bcrypt almacena contraseñas de forma segura.

Base de datos	PostgreSQL	Base relacional robusta, ideal para usuarios, animes, episodios e historial.
ORM	Prisma	Simplifica consultas, migraciones y validación de relaciones.
Infraestructura	Vercel (frontend) + Render/Railway (API y BD)	Despliegue simple y suficiente para unos 15 usuarios.
Archivos multimedia	URLs de proveedor autorizado/CDN	Evita almacenar vídeos pesados directamente en el servidor.

3. Modelo de datos antiguo

User

- id: UUID, clave primaria.
- name: nombre del usuario.
- email: único.
- passwordHash: contraseña cifrada con bcrypt.
- createdAt, updatedAt.

Anime

- id: UUID, clave primaria.
- title: título.
- description: sinopsis.
- coverUrl: imagen de portada.
- genre: género o lista de géneros.
- releaseYear: año de estreno.
- status: en emisión, finalizado, próximo.
- createdAt, updatedAt.

Season

- id: UUID.
- animeId: referencia a Anime.
- number: número de temporada.
- title: nombre opcional.

Episode

- id: UUID.
- seasonId: referencia a Season.
- number: número del capítulo.
- title: título.
- description: descripción opcional.
- durationSeconds: duración.
- videoUrl: URL del proveedor autorizado.
- thumbnailUrl: miniatura.
- publishedAt: fecha de publicación.

WatchHistory

- id: UUID.
- userId: referencia a User.
- episodeId: referencia a Episode.
- progressSeconds: segundo exacto de reproducción.
- completed: indica si terminó el capítulo.
- lastWatchedAt: última visualización.

3. Modelo de datos Nuevo

User

- id: UUID, clave primaria.

- name: nombre del usuario.
- email: único.
- passwordHash: contraseña cifrada con **bcrypt**.
- createdAt: fecha de registro.
- updatedAt: última actualización.
- history: relación con el historial de reproducción.

Anime

- id: UUID, clave primaria.
- title: nombre de la colección.
- description: sinopsis o descripción de la colección.
- coverUrl: imagen de portada.
- genre: categoría y subgéneros, separados por “.”.
- releaseYear: año de publicación de la colección.
- status: ONGOING, COMPLETED o UPCOMING.
- createdAt, updatedAt.
- seasons: relación con sus temporadas.

Season

- id: UUID, clave primaria.
- animeId: referencia a Anime.
- number: número de temporada.
- title: nombre opcional; actualmente identifica la colección/canal de práctica.
- anime: relación con Anime.
- episodes: relación con los episodios.
- Único: animeId + number.

Episode

- id: UUID, clave primaria.
- seasonId: referencia a Season.
- number: número del episodio.
- title: título adaptado del vídeo de práctica.
- description: descripción y canal de procedencia.
- durationSeconds: duración total en segundos.
- videoUrl: URL pública de YouTube.
- thumbnailUrl: miniatura del vídeo.
- publishedAt: fecha de publicación en la plataforma.
- season: relación con Season.
- history: relación con el historial de los usuarios.
- Único: seasonId + number.

WatchHistory

- id: UUID, clave primaria.
- userId: referencia a User.
- episodeId: referencia a Episode.
- progressSeconds: segundo exacto de reproducción.
- completed: indica si terminó el episodio.
- lastWatchedAt: última visualización o actualización.
- user: relación con User.
- episode: relación con Episode.
- Único: userId + episodeId.

AnimeStatus

- ONGOING: en emisión.
- COMPLETED: finalizado.
- UPCOMING: próximo.

Anime 1 — N Season
Season 1 — N Episode
User 1 — N WatchHistory
Episode 1 — N WatchHistory

Paso 1: Usuario abre la página principal

-> Paso 2: Si no tiene sesión iniciada, visualiza la landing page con información de AniTrack, episodios destacados y las opciones “Crear una cuenta” e “Iniciar sesión”

-> Paso 3: Usuario se registra o inicia sesión

-> Paso 4: El backend valida los datos y genera un JWT con vigencia de 3 horas

-> Paso 5: React guarda el token, los datos básicos del usuario y la hora de su última actividad en localStorage

-> Paso 6: Usuario autenticado accede al catálogo principal

-> Paso 7: Puede abrir “Biblioteca” desde el header o el menú lateral móvil

-> Paso 8: Filtra episodios por categoría, fecha de publicación, subgénero o colección

-> Paso 9: Selecciona una colección para abrir el detalle, su temporada y el episodio deseado

-> Paso 10: El sistema verifica que exista una sesión válida antes de permitir el acceso al reproductor

-> Paso 11: Usuario reproduce un capítulo mediante el proveedor de vídeo autorizado

-> Paso 12: En vídeos HTML5, el reproductor envía periódicamente el progreso al backend

-> Paso 13: El backend crea o actualiza WatchHistory para ese usuario y episodio

-> Paso 14: Usuario abre “Mi historial” desde el header o el menú lateral

-> Paso 15: El sistema muestra los episodios vistos y permite reanudar desde el último segundo guardado

-> Paso 16: Cada interacción del usuario mantiene activa la sesión y renueva el token periódicamente

-> Paso 17: Si no hay actividad durante 3 horas, React elimina la sesión automáticamente y devuelve al usuario a las opciones de acceso

5. Decisiones arquitectónicas principales

1. Monolito modular con React y Express

Para 15 usuarios, separar microservicios añade complejidad sin aportar valor real.

2. API REST

Es simple de implementar, probar y consumir desde React para recursos como usuarios, animes, episodios e historial.

3. PostgreSQL como fuente principal de datos

Las relaciones entre usuarios, animes, temporadas, episodios e historial se modelan claramente en una base relacional.

4. Historial desacoplado de episodios

El progreso pertenece al usuario, no al episodio; WatchHistory permite reanudar vídeos individualmente.

5. Vídeos fuera de la infraestructura principal

El backend guarda referencias (videoUrl) y no archivos pesados, reduciendo costes y problemas de rendimiento.

NotebookLM's

The Complete Node.js, Express, and MongoDB Backend Guide

<https://notebook.google.com/notebook/f90504b4-aaf2-4d26-a900-ef893585794c>

Fundamentos de HTML y CSS: Curso de Desarrollo Web CTF

<https://notebook.google.com/notebook/3c9a11d9-5ea2-4d63-8840-d1826a9fae48>